Numpy

>>> import numpy as np

arr1 = np.array([1,2,3,4,5])

print(arr1) → [1 2 3 4 5]

print(type(arr1)) → <class 'numpy.ndarray'>

print(arr1.shape) → (5,) → 1 dimension

>>> a → array([1,2,3,4,5])

→ Reshaping an array

>>> arr2 = np.array([1,2,3,4,5])

✗ arr2.reshape(1,5) → 1 row & 5 columns

→ array([[1,2,3,4,5]])

2d array, 2 brackets

but arr2 ⇒ [1 2 3 4 5]

✓ arr2 = arr2.reshape(1,5)

2d array now

new dimensions

Note → print(arr) → prints without commas.

also

a = a.reshape(5,1)

array → simply print array([5 9 2 1])

⇒ a ⇒ [[1][2][3][4][5]] — 5 rows, 1 column

range step reshape 5 rows 1 col

(*) np.arange(0, 10, 2).reshape(5, 1)

```
=> array([[0],
          [2],
          [4],
          [6],
          [8]])
```

(*) np.ones((3, 4)) → to create 3 rows 4 columns of 1's

```
=> array([[1., 1., 1., 1.],
          [1., 1., 1., 1.],
          [1., 1., 1., 1.]])
```

(*) identity matrix → (diagonal elements are 1, rest 0)

np.eye(3)

```
=> array([[1., 0., 0.],
          [0., 1., 0.],
          [0., 0., 1.]])
```

(*) More inbuilt array functions. (*)(*)

- → print('Array', arr) → prints the array
- → arr.shape → dimensions
- → arr.ndim → number of dimensions
- → arr.size → size / total elements in array
- → arr.dtype → data type in array
- ~~→ arr.bytesize~~
- → arr.itemsize → itemsize in bytes

more

Inbuilt ~~forms~~ attributes of np.

arr → ~~[illegible]~~ returns the array.

arr.shape → returns the dimensions.

arr.ndim → returns the no. of dimensions

arr.size → returns the size (total no of elements)

arr.dtype → ~~rets~~ returns the data type

arr.itemsize → returns itemsize in bytes.

Numpy Vectorised Operations

```
a1 = np.array([1,2,3,4])
a2 = np.array([10,20,30,40])
```

Element wise addition.

>>> print(arr1 + arr2) ⟹ [11 22 33 44]

⟹ similarly, subtraction, division, multiplication.

>>> arr1 - arr2 ⟹ [-9 -18 -27 -36]

>>> arr1 * arr2 ⟹ [10 40 90 160]

>>> arr1 / arr2 ⟹ [0.1 0.1 0.1 0.1]

Note → if we do this in list, they will get concatenated.

Universal functions (get applied element-wise)

~~(1) Square root~~

a = np.array([2,3,4,5,6])

(1) Square root

np.sqrt(a)

(3) Sine

np.sin(a)

(2) Exponential

np.exp(a)

(4) Natural log

np.log(a)

Array slicing & indexing.

a = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])

>>> a[0][0] → 1

>>> a[0] ⟹ 1st row ⟹ array([1,2,3,4])

>>> a[1:] ⟹ 2nd & 3rd row.

a[1:, 2:] → gives array([[7, 8], [11, 12]])

a[1:, 2:] VV imp

1: → 2nd row to end

2: → 3rd column to end

eg ⇒ Array: [[1 2 3 4]
[5 6 7 8]
[9 10 11 12]]

?

>>> ~~array~~ arr[1:, 1:3]

⇒ [[6 7]
[10 11]]

Modify array elements.

>>> a[0, 0] = 100 → 1st element = 100

>>> a[1:] = 100 → changes all rows ~~after~~ from 2nd to 100.

Statistical operations

>>> mean = np.mean(a)
std_dev = np.std(a)

normalised_data = (a − mean) / std_dev

Note: (normalised data is when mean = 0, std = 1)

>>> median = np.median(a)
variance = np.var(a)

imp

Logical operations

```
>>> data = np.array([1,2,3,4,5,6,7,8,9,10])
```

```
>>> data > 5
```

→ array([False, Fal. . . . . , True, True])

or

```
>>> data [data > 5]
```

indexing

→ to fetch ~~dat~~ elements > 5.

```
>>> data [(data >= 5) & (data <= 8)]
```

→ elements >= 5 & ≤ 8

→ array([5, 6, 7, 8])

logical operation only uses symbols.

& , | , > , < , ==